

Towards Reliable Hematological Diagnosis: Deep Learning for Bone Marrow Cell Classification

Francisco Neves da Silva

Chalmers University of Technology

Project 1: Mandatory Part, Advanced Part and Theoretical Questions

Student ID: 20030225-T277

Abstract—The accurate classification of bone marrow cells plays an essential role in the diagnosis and treatment of hematological disorders. Traditionally, this task has relied on manual examination under a microscope by skilled specialists, a process that is not only time-intensive but also susceptible to human error and inter-observer variability. Given the complexity and morphological diversity of bone marrow cells, there is a pressing need for a reliable, efficient, and automated approach to support diagnostic workflows. In this study, I propose a Deep Neural Network (DNN) system to automate the classification of bone marrow cells, aiming to enhance diagnostic accuracy while significantly reducing the time and effort required. Hence, a Convolutional Neural Network (CNN) architecture is proposed to overcome some of these challenges. The baseline CNN model consists of a convolutional block comprising four convolutional layers, followed by five fully connected layers, each incorporating L2 regularization on both the kernel and bias terms. To enhance the performance of this baseline architecture, several modifications will be investigated, including data augmentation during the image preprocessing stage, the incorporation of batch normalization and the addition of dropout layers. The effectiveness of each of these improvements will be systematically evaluated through an ablation study. Finally, an out-of-distribution (OOD) detection method will be applied to verify the model's ability to discriminate between in-distribution (ID) and OOD data. An automatic histogram-based thresholding method guided by kernel density estimation (KDE) is implemented to correctly identify the ID and OOD classes based on the Mahalanobis distance score. Firstly, our CNN model was trained on a dataset of 56,000 images, split into training, validation and testing sets. The best-performing model, which included batch normalization and dropout layers but excluded data augmentation, achieved a test accuracy of 82.02% and a recall of 81.01%. It also reached an AUC of 0.98 for the erythroblast (EBO), promyelocyte (PME), and plasma cell (PLM) classes. The highest precision of 90.30% and an F1-score of 88.39% were recorded for the EBO class by this model. Finally, the OOD detection model determined a threshold of 34.61, which effectively distinguished between ID and OOD data, forming two distinct hills. Some overlap was observed, likely due to interactions within the layer structures of the best-performing model and its less-than-perfect prediction quality.

Index Terms—Deep learning, Ablation Study, OOD Detection, Bone Marrow Cells, Image Classification

I. METHODOLOGY

In this study, I used the proposed methodology illustrated in Figure 1 to effectively distinguish between eight distinct categories of bone marrow cells. This section is divided into four distinct parts: Data Description and Preparation, Convo-

lutional Neural Network (CNN) Model Architecture, Out-of-Distribution (OOD) Detection and Performance Evaluation.

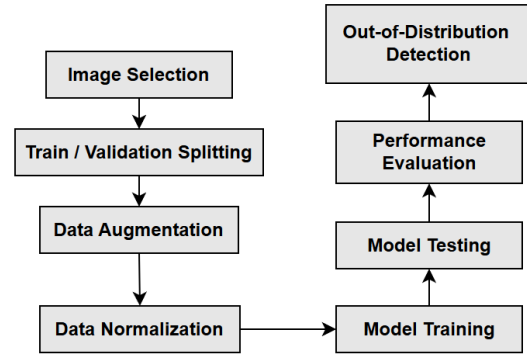


Fig. 1. Bone marrow cell classification pipeline.

A. Data Description and Preparation

The Bone Marrow Cell Classification dataset used in this study [1] comprises over 170,000 de-identified images, expertly annotated and derived from bone marrow smears of 945 individuals. These samples were stained using the May-Grünwald-Giemsa/Pappenheim method and represent a range of hematological disorders typically diagnosed in specialized leukemia laboratories. For this research, I focused on eight clinically significant cell types: Promyelocyte (PMO), Segmented Neutrophil (NGS), Band neutrophil (NGB), Plasma cell (PLM), Artefact (ART), Lymphocyte (LYT), Blast (BLA), and Erythroblast (EBO). The dataset was created by selecting a subset of the original images and resizing them, resulting in a balanced set of classes. Training and test sets were separated randomly. Of these, 32,000 images were used for training, and 24,000 images each were allocated for testing. To optimize model performance, the dataset underwent a preprocessing pipeline as shown in Figure 1.

1) **Data Augmentation:** To improve the robustness and generalization of the model, various data augmentation techniques were applied to the training dataset. These augmentations simulate different image variations and help the model perform better under diverse conditions. Specifically, the following two sets of transformations were employed randomly to the training dataset:

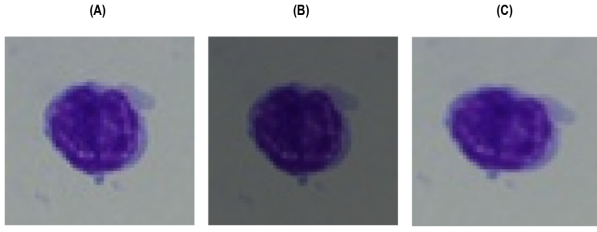


Fig. 2. Data augmentation was performed in the (A) original sample. (B) Brightness adjustment in the range of 0.5 to 1.5, along with a shear transformation up to 0.3 and (C) zooming up 30% with random rotations up to 25 degrees were the chosen transformations.

- Brightness adjustment in the range of 0.5 to 1.5, along with a shear transformation up to 0.3.
- Zooming up to 30% and random rotations up to 25 degrees.

Data augmentation was performed using the training dataset after splitting. After data augmentation, the oversampled training dataset contains a total of 76,800 images, with 51,200 new augmented images as illustrated in Figure 2. These augmentations increase the diversity of the training data and help mitigate overfitting, ultimately contributing to improved model performance. For the propose of this work, I will include only the conventional augmentation techniques despite nowadays other more efficient methods have been proposed as seen in the recent work by Kumar et al. [2], where a novel augmentation model that includes: pairwise channel transfer, novel occlusion and masking augmentation.

2) *Data Normalization*: Z-score normalization, commonly referred to as standardization, is an essential preprocessing technique in machine learning. It transforms data to conform to a standard normal distribution, ensuring that all features are measured on a uniform scale. This process prevents features with larger numeric ranges from excessively influencing the model, thereby enhancing overall performance. Thus, the pixel intensity values $\mathbf{X}$, with values from 0 to 255, can be re-scaled as follows:

$$\mathbf{Z} = \frac{\mathbf{X} - \mu}{\sigma} \quad (1)$$

where μ is the mean and σ is the standard deviation of the training dataset. By eliminating unit dependencies, Z-score normalization allows for meaningful comparison between variables measured in different units. Furthermore, it aids in identifying outliers, as values with extreme Z-scores often represent anomalies within the dataset.

B. CNN Model Architecture

In this study, it was developed a custom CNN designed to classify bone marrow cell images from diverse individuals as shown in Figure 3. The architecture consists of four convolutional layers followed by five fully connected layers. Each convolutional layer employs a 3×3 kernel, with filter sizes progressively increasing through the network as follows: 32, 64, 128 and 256. For each convolutional layer, padding

was applied evenly to the left and right or up and down of the input. Moreover, all convolutional layers include an activation function, which is essential for introducing non-linearity to the model [3]. These activation functions operate on the output of a linear transformation, i.e. a weighted sum of the inputs. Among the various activation functions, the Rectified Linear Unit (ReLU) has been widely adopted due to its simplicity and effectiveness. The ReLU activation function can be written as:

$$f(\mathbf{x}; \theta) = \max(0, \mathbf{x}) \quad (2)$$

ReLU not only speeds up training by mitigating the vanishing gradient problem but also demonstrates robustness against various types of image noise [3]. As a result, it might contribute to the improved accuracy of image classification tasks in CNN models [4]. Furthermore, average pooling will be implemented in each convolutional block, as previous studies have shown that it provides higher performance compared to both min-pooling and max-pooling architectures [5].

After the convolutional layers, a flatten layer is used to convert the multidimensional output into a one-dimensional array, making it suitable as input for the subsequent five dense layers. The final dense layer consists of eight units, corresponding to the eight output classes. A softmax activation function is applied in the final layer to generate a categorical probability distribution over these eight categories:

$$p(y | \mathbf{x}) = \frac{e^{f_y(\mathbf{x}; \theta)}}{\sum_{i=1}^K e^{f_i(\mathbf{x}; \theta)}} \quad (3)$$

where $f_y(\mathbf{x}; \theta)$ denotes the logit corresponding to the y^{th} class label. From this baseline CNN model, several improvements will be applied in an attempt to improve the model's image classification accuracy. The following improvements were included during the design of the CNN model:

1) *Batch Normalization*: Batch normalization is a widely used technique in deep learning that helps stabilize and accelerate the training process by normalizing the inputs to each layer. This normalization ensures that the input to each layer has a mean of zero and a variance of one, reducing the sensitivity of the network to the scale of the input data. As a result, batch normalization enables faster convergence by allowing the use of higher learning rates, reduces the dependence on careful weight initialization, and often leads to better generalization, helping to prevent overfitting [6]. However, when working with very small batch sizes, the computed statistics can become noisy reducing its effectiveness.

2) *Dropout Layers*: Dropout is a regularization technique used to prevent neural networks from overfitting to the training data. It works by randomly deactivating a fraction of the neurons during each training iteration, effectively forcing the model to learn more distributed and robust representations. By removing different subsets of neurons each time, dropout introduces useful randomness into the learning process, reducing the model's reliance on specific paths through the network and encouraging it to generalize better. During inference, all neurons are active, but their outputs are scaled down to account

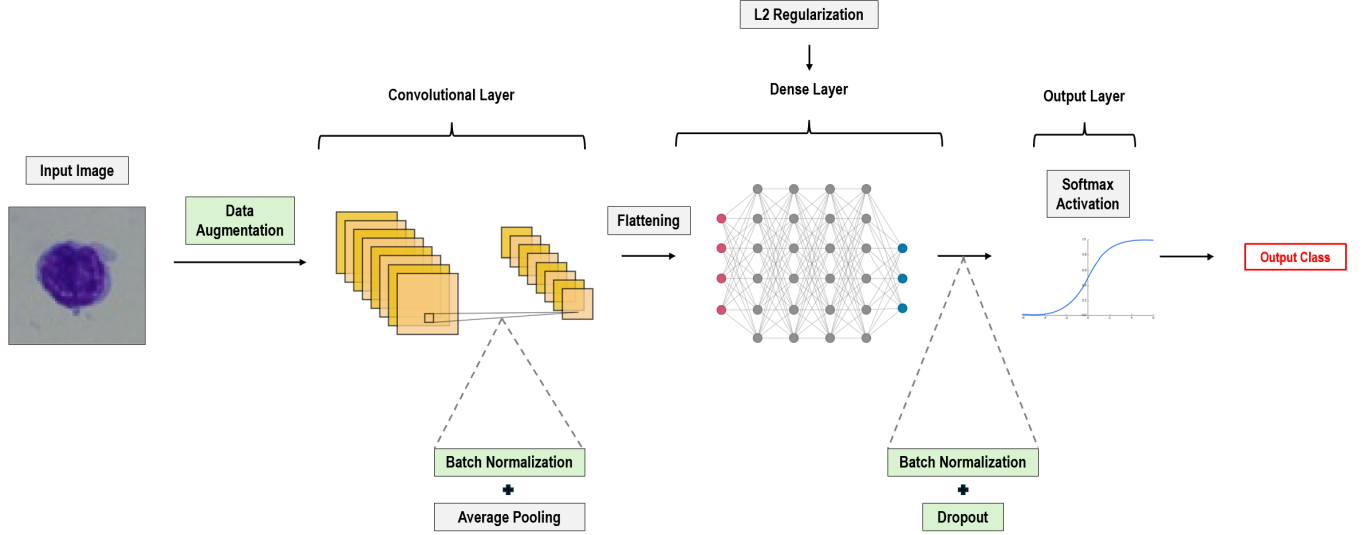


Fig. 3. A schematic illustration of the design flow of the proposed CNN model. The grey boxes represent architectural components included in the baseline model, while the green boxes highlight improvements to the baseline that will be evaluated through the ablation study.

for the dropped neurons during training ensuring consistent output magnitudes. Dropout is most commonly applied after fully connected layers, where the risk of overfitting is higher due to the large number of parameters [7].

3) *Training Parameters Tuning*: The model was refined by experimenting with various parameters, including batch sizes, epochs, callbacks and learning rates. The final CNN model included the following parameters:

TABLE II
CNN MODEL TRAINING PARAMETERS

Training Parameter	Value
Optimizer	Adam
Epochs	15
Kernel Regularizer	L2 (0.0001)
Bias Regularizer	L2 (0.0001)
Batch Size	32
Callbacks	ReduceLROnPlateau
Patience	5
Loss Function	Categorical Crossentropy
Learning Rate	0.01
Learning Rate (Minimum)	0.0000001

Furthermore, to prevent overfitting and slow convergence in the loss function during training in the baseline CNN model,

it was decided to apply weight regularization on dense layers. Regularization introduces penalties on model parameters or activations, added to the loss function during optimization and typically applied per layer. Keras supports L1 and L2 regularization: L1 encourages sparsity by penalizing the absolute values of weights, while L2 penalizes their squared values, promoting smaller weights as shown below in the loss function:

$$L(\mathbf{x}, y) = \sum_{i=1}^n (y_i - f(x_i; \theta))^2 + \lambda \sum_{i=1}^m \theta_i^2 \quad (4)$$

where n is the total number of training examples, m is the total number of parameters and λ is the regularization coefficient controlling the penalty strength. During backpropagation, these penalties adjust the gradient, discouraging large weights. A higher λ results in stronger penalization and improved generalization.

C. Out-of-Distribution Detection

After training the CNN model, as shown in Figure 1, it becomes evident that the model is capable of identifying

TABLE I
PERFORMANCE COMPARISON ACROSS ABLATION SETTINGS

Model	Ablation Settings			Evaluation Metrics				
	Data Augmentation	Dropout	Batch Normalization	Training Accuracy (%)	Test Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
A				79.18	75.91	81.19	70.82	75.38
B		✓	✓	87.77	82.01	83.27	81.10	82.17
C	✓		✓	99.27	80.54	81.35	80.00	82.28
D	✓	✓		51.31	57.04	83.98	25.03	54.63
E	✓	✓	✓	94.79	78.58	79.32	78.04	79.72

relevant patterns within the data. However, it is equally important for the model to recognize inputs that fall outside the distribution of the training data referred to as "unknowns." This highlights the importance of OOD detection in ensuring the model's reliability and robustness. Data within the problem domain is referred to as in-distribution (ID) data such as bone marrow cells in this study. This includes both the images used to train an image classification network and other images belonging to the same classes. Conversely, any image that does not belong to these classes is classified as OOD such as blood cells. Detecting OOD samples is crucial for identifying inputs that may lead to unreliable predictions by deep neural networks (DNN). The goal in OOD detection is to distinguish ID samples from OOD ones with minimal error. Lee et al. [8] introduced a well-known score $M(\mathbf{x})$ that represents the Mahalanobis distance for a test sample $\mathbf{x}$ from a class distribution expressed as:

$$M(\mathbf{x}) = \sqrt{(f(\mathbf{x}; \theta) - \hat{\mu}_c)^\top \hat{\Sigma}^{-1} (f(\mathbf{x}; \theta) - \hat{\mu}_c)} \quad (5)$$

For each class $c \in \{ID, OOD\}$, the empirical class mean $\hat{\mu}_c$ and the class covariance matrix $\hat{\Sigma}$ of the Gaussian distributions are estimated from the training data. Given a test sample $\mathbf{x}$, the Mahalanobis distance is computed as the distance between the sample and the closest class-conditional Gaussian distribution. The resulting Mahalanobis distance score reflects how well the sample conforms to the statistical profile of a given class, with smaller values indicating higher conformity and thus greater confidence in class membership, i.e. higher likelihood of being an ID sample. The value for the activation function was extracted from the fourth dense layer.

D. Performance Evaluation

Performance metrics were computed using functions from the `keras.metrics v3.10.0` module, including: accuracy, precision, F1 score and recall parameters as well as confusion matrices. Furthermore, multiclass ROC curves were extracted to assess interclass variability using `sklearn.metrics` module.

II. EXPERIMENTAL EVALUATION

After data preprocessing and designing the baseline CNN model, the performance evaluation was extracted as depicted in Section I-D. The provided dataset contains a total of 56,000 images. Initially, 80% of the training data was allocated for training, while the remaining 20% was reserved for validation during training. The final performance metrics were extracted from a separate provided test dataset.

A. Ablation Study: Model A vs Model E

The performance comparison between the baseline CNN model (Model A) and the enhanced variant with all improvements (Model E) reveals the impact of integrating data augmentation, dropout, and batch normalization in the context of bone marrow cell classification.

Model A, trained without any improvement, achieves a training accuracy of 79.18% and a test accuracy of 75.91%,

as reported in Table I. This gap between training and test accuracy already suggests some level of generalization, but the relatively low recall indicates the model's limited ability to consistently identify all relevant cell classes, especially the rarer or morphologically similar ones. This is further illustrated in the confusion matrix in Figure 4A, where noticeable misclassification (964 incorrect predictions) is observed between similar cell types, such as NGS and NGB, which can appear histologically similar due to their similarly curved nuclear shapes, both often exhibiting S or C shaped morphologies [9].

In contrast, from Table I, Model E incorporates all three techniques and shows substantial performance gains: the training accuracy rises to 94.79%, and the test accuracy improves to 78.58%. Also, the F1-score increases from 75.38% to 79.72% and recall improves by more than 7%. These improvements reflect not only a better fit to the training data, but also a more robust generalization to unseen bone marrow cell images. Furthermore, classes that were previously misclassified at a high rate by Model A showed a reduction in incorrect predictions as shown in Figure 4B, e.g., as noted earlier, between NGS and NGB (510 incorrect predictions).

These results can be largely attributed to the benefits introduced by the incorporation of these improvements illustrated in Figure 3. Firstly, through data augmentation, by applying basic transformations like rotating and adjusting brightness to the training images, the model is exposed to a broader distribution of input variations which enhances its ability to recognize cells under different orientations and imaging conditions. This synthetic diversity might mitigate overfitting, a common challenge in medical imaging where dataset sizes are often limited [10]. As a result, the model learns more invariant features, leading to improved classification performance across most cell types, as seen in the enhanced diagonal density in the confusion matrix of Model E (Figure 4B). However, one key consideration regarding the larger final dataset size after augmentation is the increased demand on memory and computational resources introduced [11].

Moreover, the use of dropout further reinforces these improvements by encouraging the network to learn redundant and distributed feature representations. By randomly deactivating neurons during training, dropout prevents the model from becoming overly reliant on specific activations, thereby fostering resilience to noise and outliers. This mechanism likely contributes to the improved recall and F1-score in Model E.

During training, the parameters of a neural network are constantly changing, which causes the distribution of activations to "shift". This phenomenon, known as internal covariate shift [6], can slow down learning because each layer has to continuously adapt to the shifting input distributions from the layer below. Batch normalization addresses this problem by normalizing the inputs to each layer so that they have a stable distribution throughout training. This normalization helps reduce internal covariate shift, making the training process more stable and efficient [6]. As a result, the network can use higher learning rates (Table II) and converges more quickly. This effect might be reflected in the higher training

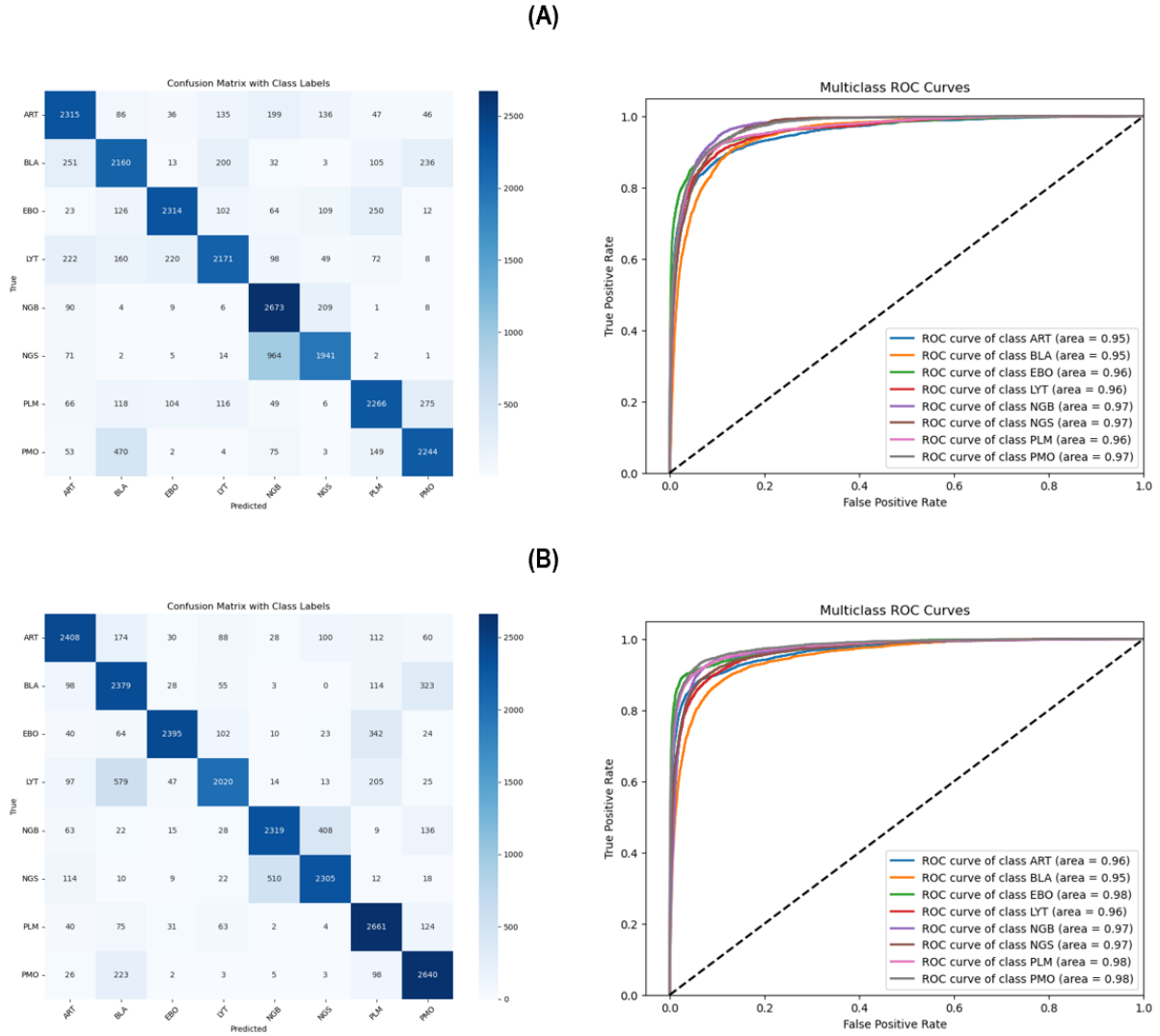


Fig. 4. Confusion matrix (left) and multiclass ROC curves (right) of predictions on testing data and generated by (A) Model A and (B) Model E.

accuracy of 94.79% achieved by Model E, compared to 79.18% for the baseline Model A, indicating faster and more stable convergence during training.

Finally, the multiclass ROC curves in Figure 4B highlight the broader improvement in the model's discriminative ability. While Model A yields area-under-curve (AUC) values between 0.95 and 0.97 across classes, Model E pushes most of these scores closer to 0.98, indicating a sharper distinction between positive and negative predictions for each class.

B. Ablation Study: Model B

Among the configurations assessed, Model B stands out as the most effective, achieving the highest test accuracy of 82.01%, alongside a recall of 81.01%, as presented in Table I. These metrics not only surpass the baseline Model A, but also outperform other models that incorporate data augmentation. Notably, Model C, which introduced data augmentation and batch normalization, reached a slightly lower test accuracy (80.54%) and a comparable F1-score (82.28%), but with a

lower recall (80.00%). While Model D achieved the highest precision (83.98%), its extremely low recall (25.03%) and resulting F1-score (54.63%) indicate significant class imbalance or underfitting issues, suggesting that the model failed to generalize properly. However, this underfitting issue persists after some epochs in Model B, since the loss for validation is higher than for training (Figure 5C).

TABLE III
CLASSIFICATION REPORT FOR MODEL B

Class	Precision (%)	Recall (%)	F1-Score (%)	Support
ART	81.09	84.20	82.61	3000
BLA	76.77	77.57	77.17	3000
EBO	90.30	86.57	88.39	3000
LYT	82.11	79.60	80.84	3000
NGB	78.76	78.63	78.70	3000
NGS	80.69	77.57	79.10	3000
PLM	86.35	84.13	85.23	3000
PMO	81.85	89.17	85.35	3000

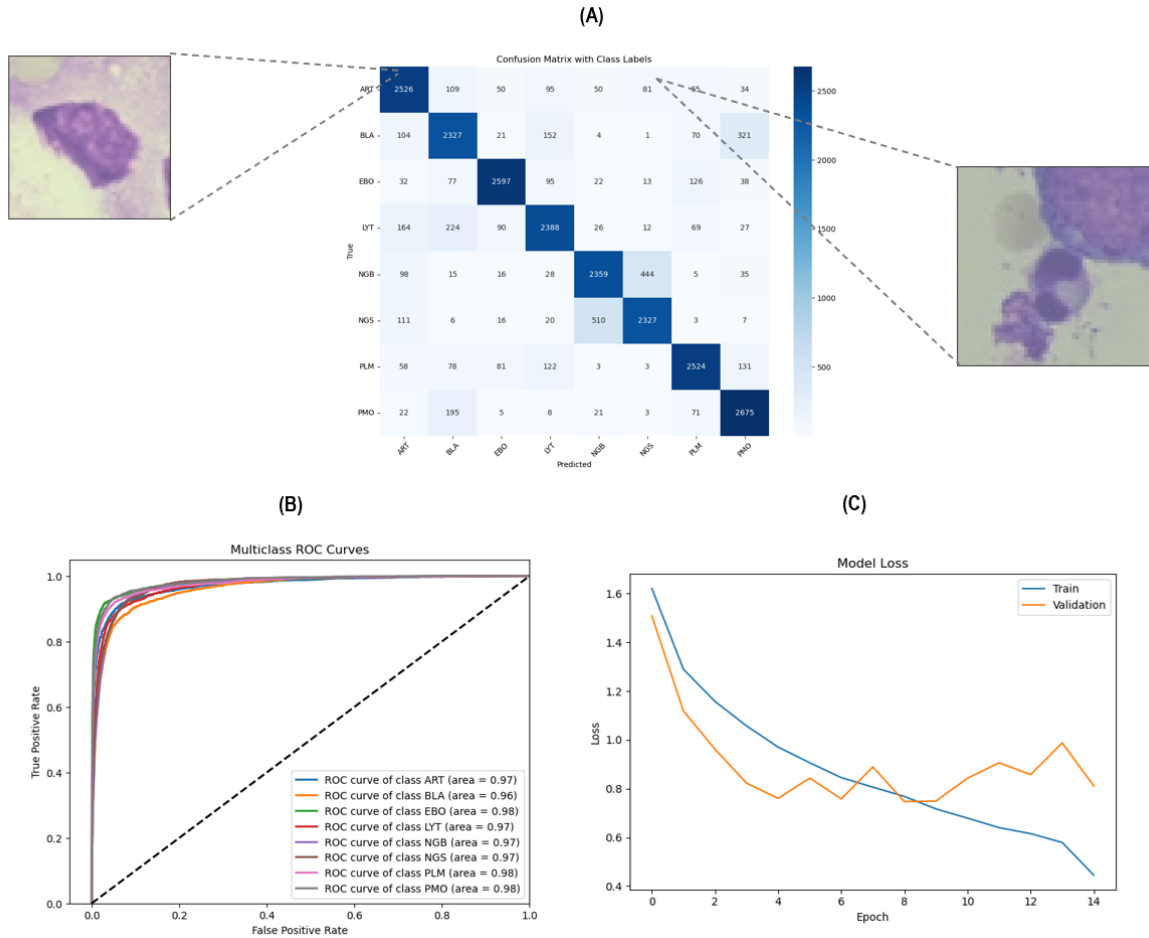


Fig. 5. Performance evaluation for Model B. (A) Confusion matrix with a correctly classified image from ART (left) and incorrectly labeled image as NGS (right) from the same class, (B) multiclass ROC curves of predictions on testing data. During training, (C) the training (blue) and validation (orange) loss functions were extracted. Model B comprises 1,024,376 total parameters: 1,023,384 trainable and 992 non-trainable parameters.

Furthermore, a possible explanation for the divergence in validation loss in Figure 5C, even with the presence of regularization in the dense layers, lies in the interaction between batch normalization and training dynamics. Batch normalization helps mitigate internal covariate shift by stabilizing layer inputs and decoupling them from parameter changes in preceding layers [6], it also allows for higher learning rates and can reduce reliance on other regularization methods such as dropout. This effect could be seen when comparing Model C and Model E: despite Model E incorporating additional regularization, its performance only marginally improved over Model C, suggesting diminishing returns when combining both strategies. However, batch normalization's effectiveness can be compromised under certain conditions. For example, in Model D, potential oversampling from data augmentation may have distorted the input distribution, amplifying the internal covariate shift leading to extremely low values for test and training accuracy. Furthermore, small batch sizes or poor representativeness of mini-batch statistics during training can destabilize learning, particularly in augmented datasets where

synthetic variability may not reflect the true population distribution [12]. Interestingly, Model A, trained on a smaller yet more stable dataset without augmentation, maintained more consistent performance, likely due to reduced distributional noise and fewer disruptions in feature space across layers [11] [12]. Additionally, the discrepancy observed between training and validation loss curves can also be attributed to the presence of regularization techniques which are only applied during training [13].

The confusion matrix in Figure 5A reveals that the model performs particularly well in leukocyte classes with distinct morphological characteristics, such as PLM (2524 correct predictions), PMO (2675 correct predictions), and EBO (2597 correct predictions), where features like lobulated nuclei, high cytoplasmic granularity [9] and clear segmentation outstand. In contrast, misclassification is more common among biologically similar classes, such as BLA and PMO, due to high nucleus-to-cytoplasm ratios, dense chromatin and round monolobulated nuclei [9]. The images in Figure 5A visually support these findings: the left image appears to show a

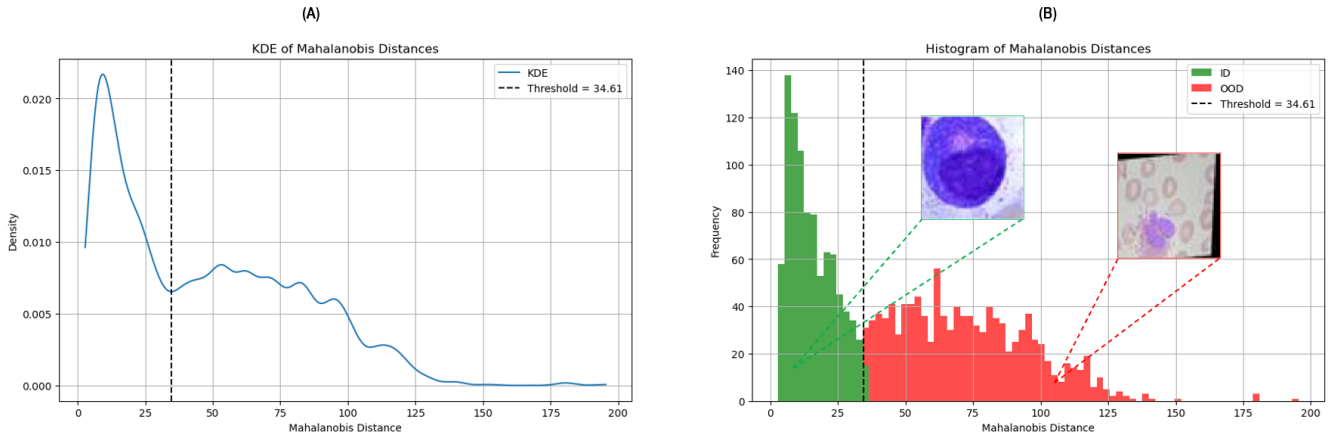


Fig. 6. OOD detection. (A) Histogram-based thresholding method guided by KDE achieving a threshold of 34.61 (black dashed line) and (B) histogram plot for ID (green) and OOD (red) samples using the Mahalanobis distance score (Equation 5). Out of the 2,000 samples, a sample from ID (left, $M(x) = 9.12$) and OOD (right, $M(x) = 114.44$) distribution were randomly selected as a verification step.

cluster of overlapping or fragmented cell debris, while the right image contains poorly centered out-of-focus elements with ambiguous cell boundaries, both of which lack definable cell features. Indeed, NGS have a multilobulated nuclei that can be easily confounded as multiple clusters of cell debris with poorly defined cell boundaries. A further source of misclassification is the presence of overlapping or multiple cell clusters within a single patch or poor centering of the target cell. Still, these qualitative insights align with the class-wise performance metrics shown in Table III, where classes like EBO and PLM achieved high F1-scores, reflecting both the model’s effective feature extraction and the clarity of their morphological traits. Moreover, the multiclass ROC curves in Figure 5B reinforce these conclusions, with AUC values between 0.96 and 0.98 across all classes.

Finally, a key challenge in deploying deep learning models in medical contexts remains the limited availability of labeled data [10], particularly for rare diseases. While data augmentation has been widely adopted as a solution to this issue, its effectiveness is mixed and context-dependent as shown by the slightly lower test accuracy values for the models containing data augmentation (Table I). Recent studies proposed more advanced approaches, such as adversarial attack driven data augmentation [14], that have shown the ability for generating diverse and realistic samples by perturbing images in ways that preserve diagnostic features, thereby enhancing the robustness of medical models. However, traditional augmentation techniques, while easy to implement and useful for rapid oversampling, can mistakenly lead to overfitting, particularly for minority classes [11] as shown in the confusion matrix for Model E in Figure 4B. This occurs because repeated synthetic variants of underrepresented samples reinforce their existing biases without truly increasing dataset diversity.

A key takeaway from the results in Table I is that the combination of explicit regularization techniques, such as L2 regularization and dropout, with data augmentation offers only

marginal improvements over training without augmentation likely because the original hyperparameters were not re-tuned for the new setup, and because regularization may limit the model’s ability to fully exploit the added variability from augmented data as previously described by Hernandez-Garcia et al. [15]. Also, most data augmentation methods assume that the training and test data share the same distribution. When this assumption fails, e.g. when testing on unseen disease subtypes, these methods often fall short [10].

C. Out-of-Distribution Detection

To distinguish ID from OOD samples, a threshold on the Mahalanobis distance score presented in Equation 5 was selected using a histogram-based thresholding method guided by kernel density estimation (KDE) [16]. This approach automatically identifies the first local minimum in the KDE curve, marking a transition from the high-density region to the low-density tail hypothesized to be associated with OOD samples. This technique avoids manual threshold selection and leverages the empirical bimodal distribution structure of the Mahalanobis scores to ensure data-driven, image classification.

Figure 6A shows the KDE of Mahalanobis distances, where the threshold is indicated as a dashed vertical line at a value of 34.61. The KDE exhibits two clearly distinct hills which strongly corroborates the presence of two underlying groups corresponding to ID and OOD samples. Thus, from Figure 6B, we might be able to qualitatively verify the effectiveness of the separation by randomly extracting two samples from each hill. The sample from the left hill (green) was confirmed to be an ID sample, while the sample from the right hill (red) corresponded to an OOD sample. Moreover, this observation aligns with the fact that ID samples tend to have lower Mahalanobis scores, as they lie closer to the center of the Gaussian-like feature distribution, as illustrated in Figure 6B.

Despite this success, the histogram reveals that the separation between ID and OOD samples is relatively limited.

One plausible explanation lies in the representation capacity of Model B from which the feature embeddings were extracted. Model B achieved a classification accuracy of 82.02% (Table I), indicating decent but not perfect discriminative performance. Naturally, the resulting feature space may not form tightly clustered Gaussian distributions for each class. Moreover, Mahalanobis distance relies on the assumption that the class-conditional feature distributions are approximately Gaussian [8]. However, batch normalization tends to reduce the natural variation between individual feature vectors by enforcing batch-level consistency. In doing so, it can compress the variance across samples within a class and make the features across different classes more similar [6]. This "flattening" effect can lead to overlapping Gaussian distributions in the embedding space, which makes it more difficult for Mahalanobis distance to have near-perfect performance when distinguishing between ID and OOD data.

Despite these limitations, the OOD detection method demonstrated its effectiveness in identifying two distinct distributions, enabling the separation of ID and OOD samples. Similar findings have been reported in prior work by Tholhuijsen et al. [17], where Mahalanobis distance maintained a certain level of overlap between ID, near-OOD and far-OOD samples on a different dataset. Several studies support the use of Mahalanobis distance for OOD detection due to its robustness in challenging scenarios [18], such as when the training data contains noisy labels or limited samples. Unlike posterior probabilities, which can falsely assign high confidence to distant or abnormal inputs [8], Mahalanobis distance provides a more reliable measure by considering the statistical structure of the feature space.

III. THEORETICAL QUESTIONS

Task 1.3.a

Answer: The primary difference between fully connected (FC) layers and convolutional layers lies in their connectivity and parameter-sharing mechanisms. FC layers are densely connected, meaning every output neuron is connected to every input neuron through a unique weight and hence, no special assumptions are needed to be made about the input. In contrast, convolutional layers use local connectivity and weight sharing [19]. Each neuron in a convolutional layer is connected only to a local spatial region of the input, defined by a convolutional kernel, and the same kernel weights are reused across different spatial locations.

FC layers are typically used at the end of convolutional neural networks for high-level reasoning [19], such as producing class scores in classification. They flatten feature maps into vectors, discarding spatial structure, a limitation for tasks requiring pixel-level precision.

On the other hand, fully convolutional networks (FCNs), omit FC layers and use only convolutional, pooling and possibly upsampling layers. By preserving spatial hierarchies, FCNs are well-suited for tasks like semantic segmentation [20], where outputs must align spatially with inputs. A further advantage of FCNs is their ability to handle inputs of varying

sizes, since convolutional operations are not restricted to fixed dimensions. FC layers, by comparison, require fixed-size inputs due to matrix dimensionality constraints. Although adaptive pooling can reshape feature maps for FC layers, it still reduces spatial detail making it unsuitable for dense prediction tasks [21]. For example, once spatial information is lost, it becomes impossible to recover the pixel-level precision needed for tasks like medical image segmentation. However, such a loss may be acceptable in tasks like graph classification, where spatial structure is less critical.

In summary, FC layers support global reasoning on fixed-size inputs, while FCNs preserve spatial information and support flexible input sizes. This makes FCNs ideal for dense, spatially-aware tasks such as semantic segmentation. Although FC networks make no assumptions about the input structure, they generally perform worse in vision tasks. They are less effective at feature extraction and require a significantly higher number of parameters, leading to longer training times. In contrast, FCNs are specifically designed to extract meaningful features from images using fewer parameters, making them more efficient and better suited for visual recognition tasks.

Task 1.3.b: i)

Answer: In Variational Autoencoders (VAEs), we model the data distribution $p(x)$ using latent variables z , where x represents an observation and z is the latent space. We specify a prior over the latent space, typically a standard Gaussian $p(z) = \mathcal{N}(0, I)$, and a likelihood $p(x|z)$ modeled by a neural network decoder that defines how data is generated from the latent variables. Since the true posterior distribution $p(z|x)$ is hard to compute due to the complexity of the model, we introduce an approximate posterior $q(z|x)$, parameterized by another neural network, often producing a Gaussian $q(z|x) = \mathcal{N}(\mu(x), \sigma^2(x))$, with mean and variance predicted by the encoder.

The marginal likelihood of the data is given by:

$$\log p(x) = \log \int p(x, z) dz,$$

where $p(x, z) = p(x|z)p(z)$. However, this integral is generally hard to compute for high-dimensional latent variables, making direct optimization of $\log(p(x))$ impractical. To address this, we instead optimize a lower bound on the log-likelihood known as the Evidence Lower Bound (ELBO) [22], i.e. the objective function that balances reconstruction accuracy and regularization, which can be written as:

$$\mathcal{L}(x) = \mathbb{E}_{q(z|x)}[\log p(x|z)] - D_{\text{KL}}(q(z|x) \parallel p(z)),$$

where the first term $\mathbb{E}_{q(z|x)}[\log p(x|z)]$ is the expected log-likelihood under the approximate posterior and quantifies how well the decoder reconstructs x from z , also known as the reconstruction loss, to enforce structure on the latent space. The second term is the Kullback–Leibler (KL) divergence between the approximate posterior and the prior, acting as a regularizer that penalizes deviations of $q(z|x)$ from the prior $p(z)$, thereby encouraging smooth and generalizable

latent representations. Naturally, this represents the similarity between learned and prior distribution. Unlike classical maximum likelihood estimation, where the model likelihood $p(x)$ is fixed, in VAEs the decoder $p(x|z)$ is itself a learned function. Therefore, optimizing the ELBO improves both the approximate inference model (encoder) and the generative model (decoder). Although $\log p(x)$ remains intractable, maximizing the ELBO simultaneously increases $\log p(x)$ and minimizes the divergence between the true posterior and the approximate posterior [23]. This dual objective reflects the essence of the VAE: learning a generative model while also learning an inference mechanism.

In summary, VAEs optimize the ELBO because it is a tractable and meaningful surrogate to the intractable data log-likelihood. The ELBO decomposes into an interpretable reconstruction term and a regularization term, both of which are computable through stochastic gradient descent and back-propagation. This formulation allows VAEs to learn expressive probabilistic models that can generate new samples and infer structured latent variables, while avoiding the computational difficulties of directly evaluating $\log p(x)$.

Task 1.3.b: ii)

Answer: A VAE is an explicit latent variable model. It introduces a latent space, typically modeled by a prior distribution such as a standard multivariate Gaussian $p(z) = \mathcal{N}(0, I)$. The encoding process in a VAE maps an input data point x to a distribution over latent variables $q(z|x)$, often parameterized as a diagonal Gaussian $\mathcal{N}(\mu(x), \sigma^2(x))$, where $\mu(x)$ and $\sigma(x)$ are outputs of the encoder neural network. Sampling from this approximate posterior yields latent codes z , which are then passed through a decoder network that reconstructs x by modeling $p(x|z)$. The training objective is to maximize the ELBO, balancing reconstruction fidelity with regularization of the latent space through a KL divergence between $q(z|x)$ and $p(z)$ [23]. Thus, the latent space in VAEs plays a crucial role in controlling the generative process and enabling interpolation and representation learning.

In contrast, Diffusion Probabilistic Models do not rely on an explicit low-dimensional latent variable. Instead, they define a Markovian forward process that gradually corrupts data x into pure noise over many steps, typically Gaussian noise, and a learned reverse process that reconstructs x from noise by sequential denoising [24]. In the forward (encoding) process, Gaussian noise is added step-by-step to the data, producing a sequence $x_0, x_1, \dots, x_T$, where $x_T \sim \mathcal{N}(0, I)$ represents nearly complete noise. The reverse (decoding) process consists of learning a series of conditional distributions $p_\theta(x_{t-1}|x_t)$ that iteratively denoise the sample. Although this process can be interpreted as implicitly learning a trajectory in a high-dimensional latent space, there is no fixed latent code z like in VAEs. Instead, the noise vectors at different time steps carry latent information implicitly.

An intuitive difference lies in how data is represented and generated: VAEs compress the input into a static, low-dimensional latent vector and decode it in one shot, whereas

diffusion models spread the encoding over a high-dimensional, stochastic time trajectory and decode through iterative refinement. As a result, diffusion models often produce higher-quality samples with greater diversity but at significantly higher computational cost due to the large number of steps involved during sampling. VAEs, on the other hand, are computationally more efficient but may suffer from issues such as blurry outputs due to limitations in modeling complex data distributions with a simple latent prior.

In summary, VAEs use a structured and explicit latent space to model data generation through a one-step decode from a latent code, whereas diffusion models forego an explicit latent variable in favor of a multi-step denoising process that captures data complexity more flexibly but at a higher computational cost.

Task 1.3.c: i)

Answer: Triangulation is the process of estimating the 3D coordinates of a point $\mathbf{X} = [X, Y, Z, 1]^T$ in homogeneous coordinates, given its 2D projections $\mathbf{x}_i = [u_i, v_i, 1]^T$ in multiple calibrated images with known camera projection matrices $\mathbf{P}_i \in \mathbb{R}^{3 \times 4}$. The image formation model relates the 3D point to its projection by:

$$\mathbf{x}_i \sim \mathbf{P}_i \mathbf{X}, \quad \text{for } i = 1, \dots, k.$$

Due to the projective ambiguity, the exact 3D position cannot be recovered from a single image, but multiple views constrain the location of $\mathbf{X}$ in space. Using Linear Triangulation [25], we can derive a solvable system. Firstly, we express the projection constraint using the cross product:

$$\mathbf{x}_i \times (\mathbf{P}_i \mathbf{X}) = \mathbf{0},$$

which eliminates the unknown scale in $\lambda \mathbf{x} = \mathbf{P} \mathbf{X}$ and yields a set of linear equations. Writing $\mathbf{P}_i$ as:

$$\mathbf{P}_i = \begin{bmatrix} \mathbf{p}_{i1}^T \\ \mathbf{p}_{i2}^T \\ \mathbf{p}_{i3}^T \end{bmatrix}, \quad \text{with } \mathbf{p}_{ij} \in \mathbb{R}^4,$$

we substitute the projection into the cross product and extract two independent equations per view:

$$u_i(\mathbf{p}_{i3}^T \mathbf{X}) - (\mathbf{p}_{i1}^T \mathbf{X}) = 0, \quad v_i(\mathbf{p}_{i3}^T \mathbf{X}) - (\mathbf{p}_{i2}^T \mathbf{X}) = 0.$$

Thus, forming two lines for each 2D point correspondence [26]. Indeed, one 2D to 3D point correspondence can result in two equations. Stacking all such equations from all views yields a linear system of the form:

$$\mathbf{A} \mathbf{X} = \mathbf{0},$$

where $\mathbf{A} \in \mathbb{R}^{2k \times 4}$ accumulates constraints from k views. This homogeneous linear system is solved using Singular Value Decomposition (SVD). The optimal estimate of $\mathbf{X}$ minimizes $\|\mathbf{A} \mathbf{X}\|$ subject to $\|\mathbf{X}\| = 1$, and corresponds to the right singular vector of $\mathbf{A}$ associated with the smallest

singular value. After computing the homogeneous solution $\mathbf{X} = [X, Y, Z, W]^T$, the final 3D point in Euclidean coordinates is obtained via dehomogenization:

$$[X, Y, Z]^T = \frac{1}{W}[X, Y, Z]^T.$$

This linear triangulation method assumes ideal conditions and can extend to multiple views [25]. In practice, due to image noise and calibration errors, it is often refined by minimizing the reprojection error across all views using non-linear optimization techniques like bundle adjustment [27] or minimizing sum of squared errors.

Task 1.3.c: ii)

Answer: In the case of two images ($k = 2$) taken from calibrated cameras with known relative pose, we are given corresponding 2D points $\mathbf{x}_1$ in the first image and $\mathbf{x}_2$ in the second. Each 2D observation defines a ray in 3D space along which the corresponding 3D point must lie. These rays are back-projected from the camera centers through the image points using the respective camera projection matrices $\mathbf{P}_1$ and $\mathbf{P}_2$, such that the 3D point $\mathbf{X}$ satisfies:

$$\mathbf{x}_1 \sim \mathbf{P}_1 \mathbf{X}, \quad \mathbf{x}_2 \sim \mathbf{P}_2 \mathbf{X}.$$

Under general conditions, these rays intersect at a unique point $\mathbf{X}$ in space [25], which can be computed via triangulation by solving a system of linear equations derived from the cross-product constraint $\mathbf{x}_i \times (\mathbf{P}_i \mathbf{X}) = 0$ for $i = 1, 2$. However, there exists a specific geometric configuration in which infinitely many 3D points may satisfy these projection constraints. This ambiguity arises when the viewing rays corresponding to $\mathbf{x}_1$ and $\mathbf{x}_2$ are coplanar and intersect along a common line in 3D, such as the baseline connecting the optical centers of two cameras [28]. A classic example of this is a critical configuration, in which both camera centers and the 3D point are collinear. In such a case, all 3D points along that line will project to the same 2D correspondences in both images due to the alignment of epipolar geometry. The epipolar constraint implies that:

$$\mathbf{x}_2^\top \mathbf{F} \mathbf{x}_1 = 0,$$

where $\mathbf{F}$ is the fundamental matrix encoding the relative geometry between the views. While this condition ensures that $\mathbf{x}_1$ and $\mathbf{x}_2$ correspond to the same 3D point up to epipolar geometry, it does not uniquely determine the position of $\mathbf{X}$. When the rays are nearly parallel or lie on the same plane, e.g. due to a short baseline or a very distant scene, the triangulation becomes ill-conditioned, and depth estimates become unreliable.

Therefore, although $\mathbf{x}_1$ and $\mathbf{x}_2$ are observable, the true 3D location $\mathbf{X}$ may not be uniquely recoverable. In these degenerate cases, an infinite number of 3D points project to the same 2D observations, and without further constraints, such as additional camera views, prior knowledge of the scene structure, or temporal consistency [28], deriving a unique solution is impossible.

USE OF LLMs

Throughout the development of this project, LLMs, such as ChatGPT, were employed as supportive tools to enhance both the scientific writing and coding aspects of the work. Their contributions are described below.

Mandatory Part

- LLMs were used to improve the quality of scientific writing by correcting grammatical, syntactic and orthographic errors, resulting in clearer and more professional text. In the coding portion, they provided support in cleaning the code to enhance accessibility and overall functionality.
- Guidance was offered in handling image loading, specifically by clarifying how the function `create_df_from_folder` could be used to extract image data directly from the local environment, simplifying dataset construction and preparation.
- Assistance was provided in understanding the role of label encoding within the `load_images_and_labels` function. The LLM helped clarify the structure of the input labels, their transformation into a suitable format for model training, and their conversion to one-hot encoding for use with the categorical cross-entropy loss function during model compilation.
- Clarification was requested when defining the input of the model using a `Sequential` architecture. In the context of OOD detection, the model initially failed to recognize the input layer properly to get the activation function for each layer. The LLM provided guidance on how to define the input structure so that it would be correctly interpreted by the model.
- When constructing multiclass ROC curves, the use of `to_categorical` introduced a mismatch with the requirements of the plotting functions. The LLM clarified the need to apply `label_binarize` for compatibility and provided step-by-step debugging suggestions to ensure proper handling and visualization of performance metrics.
- The LLM supported the design and organization of the ablation study loop. It assisted in consolidating previously developed components into a modular structure, enabling the automated evaluation of various ablation configurations.
- The LLM contributed to a deeper understanding of how batch normalization affects internal covariate shift during training, and how this normalization can influence model convergence and the distribution of feature space.

Advanced Part

- The LLM recommended the use of `StandardScaler` from `sklearn.preprocessing` to compute the Mahalanobis distance score from the feature vectors extracted from the pretrained CNN model. It also provided guidance on how to access and retrieve these feature vectors from intermediate layers of the model.

- Clarification was provided on the use of `gaussian_kde` from `scipy.stats` to estimate the probability density function of the feature distributions. This was applied in the context of determining a threshold for OOD detection based on histogram analysis.
- The LLM assisted in understanding the fundamental concepts and theoretical background underlying OOD detection in the context of image classification. This included the motivation for distinguishing between ID and OOD samples to improve model reliability.
- Based on suggestions from the LLM, regularization of the covariance matrix was introduced in the computation of the Mahalanobis distance to ensure numerical stability, especially when dealing with high-dimensional feature vectors and limited sample sizes.
- Support was provided in analyzing the impact of batch normalization on the overlap between ID and OOD distributions as visualized in histogram plots. This helped to interpret how normalization layers can affect the separability of features for OOD detection tasks.

Theoretical Questions

- The LLM provided assistance in scientific writing and in the presentation of mathematical formulations, including guidance on how to clearly typeset and structure equations in LaTeX to ensure proper formatting and readability.
- For Task 1.3.a, support was provided to intuitively understand concepts such as the fixed input size requirement in FC layers after adaptive pooling. This was explained in terms of matrix dimensionality constraints and compatibility between convolutional outputs and dense layer inputs.
- For Task 1.3.b.i, clarification was provided on the rationale behind imposing a prior distribution over the latent space in a VAE, typically modeled as a standard Gaussian $p(z) = \mathcal{N}(0, I)$. For Task 1.3.b.ii, the LLM also contributed to a clearer understanding of the Markovian forward process in diffusion probabilistic models from a mathematical standpoint.
- For Task 1.3.c.ii, the LLM was helpful in visualizing and explaining the impact of the baseline distance between two cameras on triangulation ambiguity. It provided insight into how a narrow baseline contributes to critical configurations that affect depth estimation and 3D reconstruction.
- The LLM was especially helpful in clarifying essential concepts discussed in Chapter 22 of [28], improving overall comprehension and supporting further reading on the topic.

REFERENCES

- [1] C. Matek, S. Krappe, C. Münzenmayer, T. Haferlach, and C. Marr, "An expert-annotated dataset of bone marrow cytology in hematologic malignancies," 2021.
- [2] S. Kumar, P. Asiamah, O. Jolaoso, and U. Esiowu, "Enhancing image classification with augmentation: Data augmentation techniques for improved image classification," 2025.
- [3] K. W. Goh, S. Surono, M. Y. F. Afiatin, K. R. Mahmudah, N. Irsalinda, M. Chaimanee, and C. W. Onn, "Comparison of activation functions in convolutional neural network for poisson noisy image classification," *Emerg. Sci. J.*, vol. 8, no. 2, pp. 592–602, Apr. 2024.
- [4] Y. Jiang, J. Xie, and D. Zhang, "An adaptive offset activation function for CNN image classification tasks," *Electronics (Basel)*, vol. 11, no. 22, p. 3799, Nov. 2022.
- [5] A. S. Abeed, A. Atiq, A. A. Anjum, A. Ahmed Efat, and D. Z. Karim, "Bomacnet: A convolutional neural network model to detect bone marrow cell cytology," in *2022 25th International Conference on Computer and Information Technology (ICCIT)*, 2022, pp. 686–691.
- [6] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," 2015.
- [7] H. Wu and X. Gu, "Towards dropout training for convolutional neural networks," 2015.
- [8] K. Lee, K. Lee, H. Lee, and J. Shin, "A simple unified framework for detecting out-of-distribution samples and adversarial attacks," in *Advances in Neural Information Processing Systems*, S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, Eds., vol. 31. Curran Associates, Inc., 2018.
- [9] G. S. Travlos, "Normal structure, function, and histology of the bone marrow," *Toxicol. Pathol.*, vol. 34, no. 5, pp. 548–565, 2006.
- [10] T. Islam, M. S. Hafiz, J. R. Jim, M. M. Kabir, and M. F. Mridha, "A systematic review of deep learning data augmentation in medical imaging: Recent advances and future research directions," *Healthcare Analytics*, vol. 5, no. 100340, p. 100340, Jun. 2024.
- [11] C. Shorten and T. M. Khoshgoftaar, "A survey on image data augmentation for deep learning," *J. Big Data*, vol. 6, no. 1, Dec. 2019.
- [12] P. M. Radiuk, "Impact of training set batch size on the performance of convolutional neural networks for diverse datasets," *Inf. Technol. Manag. Sci.*, vol. 20, no. 1, Jan. 2017.
- [13] S. A. Saleh, "Why my training loss is higher than validation loss? is the reported loss even accurate?" <https://siddiqueabusaleh.medium.com/why-my-training-loss-is-higher-than-validation-loss-is-the-reported-loss-even-accurate-8843e14a0756>, 2021, accessed: 2025-05-30.
- [14] M. T. Pervin, L. Tao, and A. Huq, "Adversarial attack driven data augmentation for medical images," *Int. J. Electr. Comput. Eng. (IJECE)*, vol. 13, no. 6, p. 6285, Dec. 2023.
- [15] A. Hernández-García and P. König, "Data augmentation instead of explicit regularization," 2018.
- [16] J. Lindblad, "Histogram thresholding using kernel density estimates," in *Proceedings of the Swedish Society for Automated Image Analysis (SSAB) Symposium on Image Analysis*. Halmstad, Sweden: Uppsala University, 2000, pp. 41–44.
- [17] P. T. C. M. Tholhuijsen, "Understanding ood detection methods for image classification networks," Master's thesis, Eindhoven University of Technology, June 2024, student thesis.
- [18] S. Lu, Y. Wang, L. Sheng, A. Zheng, L. He, and J. Liang, "Recent advances in ood detection: Problems and approaches," 2024.
- [19] A. Karpathy, J. Johnson, and F.-F. Li, "Cs231n convolutional neural networks for visual recognition," <https://cs231n.github.io/convolutional-networks/add>, 2016, accessed: 2025-05-28.
- [20] R. Yamashita, M. Nishio, R. K. G. Do, and K. Togashi, "Convolutional neural networks: an overview and application in radiology," *Insights Imaging*, vol. 9, no. 4, pp. 611–629, Aug. 2018.
- [21] R. Nirthika, S. Manivannan, A. Ramanan, and R. Wang, "Pooling in convolutional neural networks for medical image analysis: a survey and an empirical study," *Neural Comput. Appl.*, vol. 34, no. 7, pp. 5321–5347, Feb. 2022.
- [22] A. Asperti and V. Tonelli, "Comparing the latent space of generative models," *Neural Comput. Appl.*, vol. 35, no. 4, pp. 3155–3172, Feb. 2023.
- [23] N. Baklazhenko, M. Conner, D. Vallmánya, and D. Wielath, "Understanding the elbo in vaes and β -vaes," March 2023, accessed: [Insert access date here].
- [24] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," 2020.
- [25] D. Murray, "C18 computer vision," <http://www.robots.ox.ac.uk/dwm/Courses/4CV/>, 2015, michaelmas

Term, University of Oxford. [Online]. Available:
<http://www.robots.ox.ac.uk/dwm/Courses/4CV>

- [26] K. Kitani, "Triangulation, 16-385 computer vision," 2023, lecture material.
- [27] B. Triggs, A. Zisserman, and R. Szeliski, "Vision algorithms '99," in *Vision Algorithms: Theory and Practice*, ser. Lecture Notes in Computer Science, B. Triggs, A. Zisserman, and R. Szeliski, Eds. Berlin, Heidelberg: Springer-Verlag, 2000, vol. 1883, pp. 298–372.
- [28] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge, England: Cambridge University Press, Jan. 2011, ch. 22, pp. 533–558.